IE 303 – Modeling and Methods in Optimization

The Computing Project

Utku Kuzey Ergül ✉

Bilkent University
Department of Industrial Engineering
Spring 2025-2026

Introduction and Motivation

This computing project accounts for 10% of your overall course grade! While formulating optimization models is inherently challenging, solving them computationally presents its own distinct set of hurdles, particularly when integer variables are involved. The objective of this assignment is to familiarize you with state-of-the-art optimization algorithms and solvers, specifically `gurobipy` (Gurobi Optimization Python API).

This assignment may be completed in *groups of two*. Prepare a **TYPED** report detailing your methodology, assumptions (if any) and final answers. Submit this report as a `.pdf` file and do not include any code (screenshots or otherwise) within the report itself. Instead, package your report along with appropriately named individual Python scripts (containing your `gurobipy` models) for each question into a single `.zip` file. Submit this archive via Moodle using the following naming convention:

`Group##_IE303_S2026_ComputingProject.zip`

A single submission per group is sufficient. Ensure that the names of all group members are clearly stated on the report. The deadline for this assignment is **08/05/2026, 23:59 (11:59PM)**. Please make sure that your report is written entirely in *your own words*. Any external resources utilized must be cited appropriately. Good luck!

> "*Nothing takes place in the world whose meaning is not that of some maximum or minimum.*"
>
> – Leonhard Euler (1707-1783)

Grading Scheme

Let $t \in \mathbb{R}_+$ denote the number of hours a submission is late. Let $\mathcal{G}, \mathcal{G}(t) \in [0, 100]$ represent your originally earned grade and your final grade after the late penalty is applied respectively. Then, your final grade will be calculated according to the following formula:

$$\mathcal{G}(t) = \begin{cases} \left\lceil \mathcal{G} e^{-\frac{t}{40 \ln 8}} \right\rceil & \text{If } 0 \leq t \leq 24 \\ 0 & \text{If } t > 24 \end{cases}$$

For instance, a submission that is exactly 24 hours late will incur a 25% penalty to the project grade. Submissions delayed by more than 24 hours will not be accepted and will automatically receive a grade of 0.

Question 0: Install `gurobipy` [10pts]

Install `gurobipy` on your computers. Note that 10% of your total project grade is allocated to the successful installation of `gurobipy` as well as the overall clarity, organization and formatting of both your code and your written report.

Question 1: Least Common Multiple [20pts]

To verify your installation of `gurobipy`, you will formulate and solve a fundamental Number Theory problem using Integer Programming (IP).

Definition 1: The *Least Common Multiple* (LCM) of a set of non-zero integers $\mathcal{D} := \{a_i\}_{i=1}^n$ is the smallest positive integer that is divisible by every element of set $\mathcal{D}$.

(a) Formulate an IP Model to find the LCM of a given set $\mathcal{D}$.

Hint: Note that the elements of set $\mathcal{D}$ are not necessarily positive! Ensure your formulation forces LCM to be positive even if some a_i are negative. This can be achieved with merely a single set of constraints!

(b) In your Python script, set the *Random Seed* to **2026** and generate a set $\mathcal{D}$ of $n = 10$ random integers in the range $[-100, 100] \setminus \{0, \pm 1\}$. You may use any random number generation method. Report your generated set $\mathcal{D}$.

(c) Using `gurobipy`, implement and solve your model from (a) using the set $\mathcal{D}$ you generated in (b). Report the optimal solution and the optimal objective function value. Verify your model by comparing its output to the result of Python's built-in LCM function.

(d) Using Python's `time` module, record the total execution time of your model from (a) (including model building, constraint generation and solving). Compare this to the execution time of Python's built-in LCM function by computing the relative difference of their runtimes $\left(\text{e.g., } \frac{|t_{\texttt{gurobipy}} - t_{\text{Python}}|}{\min\{t_{\texttt{gurobipy}}, t_{\text{Python}}\}}\right)$. Which approach is orders of magnitude faster? Explain very briefly why this significant relative difference in computational time exists.

Hint: Discussing the nature of a general-purpose optimization solver versus a dedicated mathematical algorithm would be a good answer. For that, you should first find out the specific algorithm Python's built-in LCM function uses though.

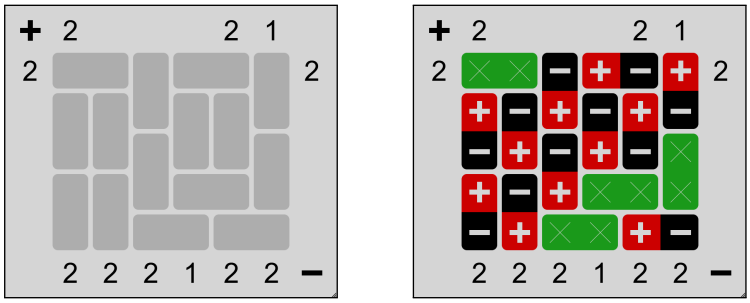
(e) **Definition 2**: The *Greatest Common Divisor* (GCD) of a set of integers $\mathcal{D} := \{a_i\}_{i=1}^n$, not all of which are zero, is the largest positive integer that divides every element of set $\mathcal{D}$.

For the case where $n = 2$, consider the set $\mathcal{D} = \{a_1, a_2\}$ where $a_1, a_2 \neq 0$. From an optimization perspective, how your model from (a) can be utilized to determine the GCD of set $\mathcal{D}$ through the below identity? Explain very briefly.

$$\text{LCM}(a_1, a_2) = \frac{|a_1 a_2|}{\text{GCD}(a_1, a_2)}$$

Question 2: The Magnetic Field Puzzle [20pts]

The *Magnetic Field Puzzle* is a logic puzzle played on a $m \times n$ grid partitioned into small rectangular blocks called *dominoes* (pairs of orthogonally adjacent cells). Each domino represents either a magnet with two charged poles, one positive ($\boxed{+}$) and one negative ($\boxed{-}$), or a "blank" domino with two neutral ($\boxed{\times}$) poles.



(a) The Puzzle Board (b) The Solution

Figure 1: A 5×6 Magnetic Field Puzzle Example

To solve this puzzle, one must determine the orientation of every domino (see Figure 1). For that, there are some external clues provided on the board: The number of $\boxed{+}$ poles in each Row / Column must match the numbers along the Top / Left moat. Similarly, the number of $\boxed{-}$ poles in each Row / Column must match the numbers along the Bottom / Right moat. Furthermore, a crucial electro-magnetic restriction applies: No two identically charged poles (i.e., two $\boxed{+}$ or two $\boxed{-}$) can be orthogonally adjacent to each other.

(a) Formulate an IP Model to find a domino arrangement for a given board. The arrangement should satisfy all properties mentioned above.

(b) Send an e-mail to the TA Utku. He will distribute randomly generated distinct Magnetic Field Puzzle instances to each group. Using `gurobipy`, implement your formulation in (a) and solve the instance provided to you.

(c) Suppose there are alternative arrangements for a given board. Can you come up with an inequality that cuts an optimal solution and does not enforce anything else? Provide an algorithm by explicitly stating each step in a way that it can be handled by a computer. You should not return a code as an answer to this question.

Question 3: Image Segmentation Using IP [25pts]

In image processing, segmentation separates a foreground object from its background. Let I be an $m \times n$ grid of pixels (i.e., an image) where each pixel $(i, j) \in I$ has a BGR color vector $\mathbf{c}^{(i,j)} \in \mathbb{R}^3$. Let $\mathbf{c}^{\text{BG}} \in \mathbb{R}^3$ denote the reference background color.

In an idealized setting, determining whether a pixel belongs to the background is trivial. However, real-world images contain noise (e.g., lighting variations, textures, etc.), blurring the mathematical boundary between foreground and background. To quantify how similar a pixel's color $\mathbf{c}^{(i,j)}$ is to the background $\mathbf{c}^{\text{BG}}$, we must select a distance metric $d\left(\mathbf{c}^{(i,j)}, \mathbf{c}^{\text{BG}}\right)$. Common distance metrics for vectors in $\mathbb{R}^n$ include:

0. (*Discrete Distance*) $d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^{n} \mathbb{1}_{\{x_i \neq y_i\}}$ where $\mathbb{1}_{\{x_i \neq y_i\}} := \begin{cases} 1 & \text{If } x_i \neq y_i. \\ 0 & \text{If } x_i = y_i. \end{cases}$
1. (*Manhattan Distance*) $d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^{n} |x_i - y_i|$
2. (*Euclidean Distance*) $d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$

∞. (*Chebyshev Distance*) $d(\mathbf{x}, \mathbf{y}) = \max_{i \in \{1,2,\ldots,n\}} |x_i - y_i|$

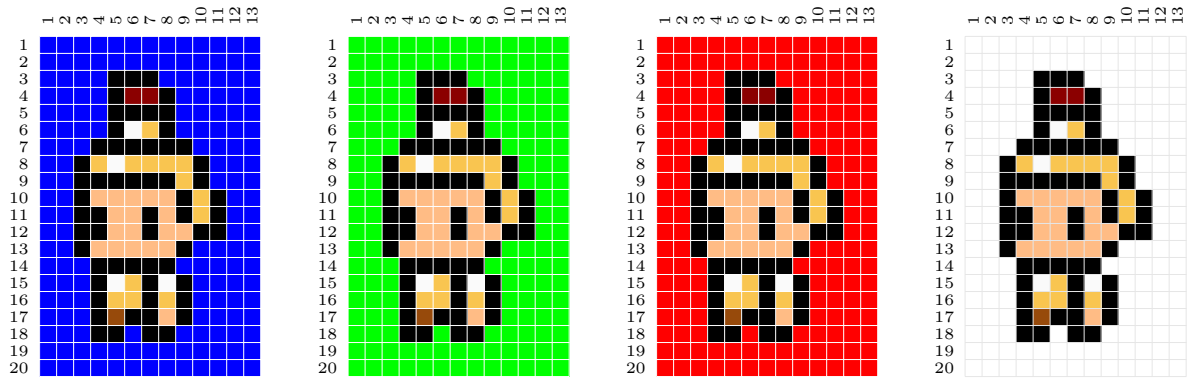
A robust segmentation model cannot rely on individual pixel colors alone! It must balance them with the "smoothness" of the overall image (i.e., neighboring pixels should ideally share the same label). To model this as an IP problem, you must design three types of penalty parameters:

- **Foreground Penalty** (f_{ij}): The penalty for labeling (i, j) as foreground. Logically, this should be **high** if $\mathbf{c}^{(i,j)}$ is close to $\mathbf{c}^{\text{BG}}$, and **low** if it is distant.
- **Background Penalty** (b_{ij}): The penalty for labeling (i, j) as background. Logically, this should be **high** if $\mathbf{c}^{(i,j)}$ is distant from $\mathbf{c}^{\text{BG}}$, and **low** if it is close.
- **Smoothness Penalty** ($s_{(i,j),(u,v)}$)**:** The penalty for separating neighboring pixels (i, j) and (u, v) by assigning them different labels. This should be **high** if their colors $\mathbf{c}^{(i,j)}$ and $\mathbf{c}^{(u,v)}$ are similar, and **low** if they are different.

To encourage creative modeling, you must construct a unique model instance. An extra 5 bonus pts will be awarded to the group that uses the most unique and mathematically sound combination of a reference background color $\mathbf{c}^{\text{BG}}$ (see Figure 2)[1], a distance metric $d(\cdot, \cdot)$ and custom mathematical functions for f_{ij}, b_{ij} and $s_{(i,j),(u,v)}$ that satisfy the logical conditions outlined above.

Hint: Because f_{ij}, b_{ij} and $s_{(i,j),(u,v)}$ are calculated before the optimization model is solved, they are simply numerical parameters in the eyes of the solver. Feel free to go all-out with non-linear or complex mathematical functions for these penalties! It will not violate the linearity of your IP.

[1] These images are very small! To view them clearly without blurring, avoid using the default "Photos" app. Instead, open them in *MS Paint*© or any other raw image editing software and zoom in.



(a) Blue Background $\mathbf{c}^{\mathrm{BG}} = \begin{bmatrix} 255 \\ 0 \\ 0 \end{bmatrix}$ (b) Green Background $\mathbf{c}^{\mathrm{BG}} = \begin{bmatrix} 0 \\ 255 \\ 0 \end{bmatrix}$ (c) Red Background $\mathbf{c}^{\mathrm{BG}} = \begin{bmatrix} 0 \\ 0 \\ 255 \end{bmatrix}$ (d) White Background $\mathbf{c}^{\mathrm{BG}} = \begin{bmatrix} 255 \\ 255 \\ 255 \end{bmatrix}$

Figure 2: Little Soldier Pixel Art Image Examples

In your implementation, you will need to extract the $m \times n$ matrix of color vectors from a provided image. You may use an open-source library such as *OpenCV* for this purpose. Furthermore, to evaluate the practical success of your mathematical model, you must calculate the Intersection over Union (IoU) metric to compare your solver's predicted segmentation mask ($\hat{\theta}$) against the ground-truth mask (θ).

$$\mathrm{IoU}(\hat{\theta}, \theta) = \frac{|\hat{\theta} \cap \theta|}{|\hat{\theta} \cup \theta|} = \frac{|\hat{\theta} \cap \theta|}{|\hat{\theta}| + |\theta| - |\hat{\theta} \cap \theta|} \in [0, 1]$$

(a) Formulate an IP Model to segment the image. Explicitly define your decision variables, define the mathematical set representing the neighborhood of a pixel, and write the objective function and constraints to minimize the total penalties.

Hint: Which classical network optimization problem taught in IE 303 does your formulation resemble? Depending on your answer, you might not need all of your decision variables to be integer! Think about it.

(b) Using `gurobipy`, implement your IP model in Python for the "Little Soldier" image (13×20 pixels) using your group's chosen unique combination. Report your chosen $\mathbf{c}^{\mathrm{BG}}$, $d(\cdot, \cdot)$, and custom penalty functions. Provide a visual representation of the resulting segmentation mask and report your calculated IoU score.

(c) Based on the Garbage In-Garbage Out (GIGO) principle and the below identity, explain very briefly how a mathematically "perfect" optimal solution can yield an "imperfect" segmentation (i.e., 0.00% Relative Optimality Gap with an IoU of < 1). Discuss how the choice of distance metric or penalty functions contributes to this phenomenon.

$$\mathbf{x}^*(\mathbf{c}, \mathbf{A}, \mathbf{b}) \in \arg\min \left\{ \mathbf{c}^\top \mathbf{x} \mid \mathbf{A}\mathbf{x} = \mathbf{b}, \mathbf{x} \in \mathbb{X} \right\}, \qquad \mathbb{X} \subseteq \mathbb{Z}_+^n$$

Question 4: Water Jug Riddle [25pts]

In the action movie *Die Hard with a Vengeance* (1995), John McClane and Zeus Carver must disarm a bomb planted by Simon Gruber in 5 minutes. To stop the countdown, they must place a jug containing exactly 4 gallons of water on a sensor. However, they are only given a 3-gallon jug, a 5-gallon jug and an unlimited supply of water from a park fountain. Because the sensor requires absolute precision, they cannot simply guess the measurements. Thus, the only reliable physical operations they can perform are:

- Fill one of the jugs completely from the fountain.
- Empty one of the jugs completely into the fountain.
- Pour water from one jug to the other until either the source jug is empty or the sink jug is completely full.

(a) Define the set of "reachable" states $\mathbb{S}$, the set of "valid" actions $\mathbb{A}(i)$ available at each state $i \in \mathbb{S}$, and the set of goal states $\mathbb{G} \subset \mathbb{S}$. Represent this state-space as a directed graph where nodes represent the "reachable" states and the edges represent the "valid" actions. Assume every "valid" action takes exactly 49 seconds to execute. In Python, starting from the initial state s (i.e., the source node), implement and execute *Dijkstra's Algorithm* on this graph. Will they disarm the bomb in time? Report the optimal sequence of actions and the total time elapsed.

Hint: Some states in the 4×6 integer grid are physically "unreachable". Uncover the underlying rule to reduce your state-space before building your graph! Secondly, it is obvious that the 3-gallon jug cannot physically hold 4 gallons of water. Deduce your goal states accordingly! Finally, assume an action is "valid" only if it changes the current state (i.e., it does not cause a self-cycle).

(b) In IE 303, you learned to solve *Sequential Decision Problems* using Dynamic Programming (DP). However, if McClane attempted to apply "standard" DP to this riddle, his logic would fail since "standard" DP requires a Finite-Horizon (i.e., a fixed number of stages) and a state-transition structure that moves strictly forward in stages, forming a *Directed Acyclic Graph*. Because the physical actions in this riddle are reversible (e.g., it is possible to fill and immediately empty a jug), the state-space contains cycles as you should have seen in (a). Hence, a "standard" DP recursion would get trapped in an infinite loop.

To solve this riddle using DP, we must treat it as an Infinite-Horizon problem under the *Stochastic Shortest Path* (SSP) framework. Please refer to the provided document, "Supplementary Note: Infinite-Horizon DP and Value Iteration", for a full explanation of how "standard" DP terms translate into the SSP framework, the formulation of the recursive equation and how to verify your solution using Linear Programming (LP).

Using the supplementary note, write down the recursive equation specifically tailored to this riddle. Implement and execute the Value Iteration (VI) algorithm in Python. Report the minimum time required to disarm the bomb and the corresponding optimal policy. Verify that your DP results perfectly match the sequence found by *Dijkstra's Algorithm* in (a).

(c) In the previous parts, you have successfully foiled Simon Gruber's plot using both Graph Theory and DP. Now, let us connect this back to the fundamental Number Theory concepts from **Question 1**.

The act of pouring water back and forth between an a_1-gallon and an a_2-gallon jug to obtain exactly b gallons of water (where $a_1, a_2, b \in \mathbb{Z}$) is mathematically equivalent to solving the *Linear Diophantine Equation*:

$$a_1 x_1 + a_2 x_2 = b$$

A fundamental theorem states that this equation has an integer solution (i.e., $x_1, x_2 \in \mathbb{Z}$) **if and only if** b is a multiple of $\text{GCD}(a_1, a_2)$.

Explain very briefly, what exactly the integer variables x_1 and x_2 represent in the context of the Water Jug Riddle. Using this conceptual link, formulate an IP Model to find $\text{GCD}(a_1, a_2)$. Solve it for $a_1 = 976$ and $a_2 = 1224$.

Hint: Reducing the problem to *Bézout's Identity*, where $b = \text{GCD}(a_1, a_2)$, would be a great starting point. Pay close attention to the direction of optimization (i.e., min vs. max) to prevent an unbounded model!